# MathLens

What it is, what it does, and how it works

A mathematics and model-analysis workbench
Version 0.2.0-alpha.1
Project overview / 28 October 2026

# The project

## Purpose

MathLens connects mathematical calculations with understandable model analysis. Users can run a method, inspect the result, and read its assumptions.

## Audience

Students, educators, developers, and curious researchers can explore bounded numerical examples without a paid AI service. The current product is a web alpha.

Core computations run deterministically. No generative AI provider is connected.

# Workspace navigation

## A consistent workspace

The sidebar connects the overview, model lab, equation explorer, historical timeline, research library, open problems, puzzles, visualizations, lessons, and documentation.

## Focused analysis views

The model lab separates regression, classification, clustering, PCA, and comparison. The workbench separates polynomials, matrices, numerical roots, definite integrals, and statistics.

# Main capabilities

## Computation and analysis

Polynomial calculus and numerical roots
Matrix operations and descriptive statistics
Regression and binary classification
Clustering, PCA, and model comparison

## Exploration and education

Gradient-descent visualizations
Curated references and a live arXiv feed
Historical timeline and open problems
Nine lessons and ten puzzle categories

# System architecture

## Browser interface

Native ES modules render semantic HTML and SVG charts. Forms send JSON to versioned APIs. Local storage holds learning progress, bookmarks, and optional experiment metadata.

## Server and providers

A Node request handler validates inputs and calls shared mathematical functions. Vercel serves static assets and a Node function. Optional Supabase handles identity. The research adapter calls arXiv.

The local Node server and the Vercel function reuse the same request handler.

# A calculation request

## Input and validation

The user enters a polynomial or a bounded matrix. The browser checks required fields. The server checks method, JSON type, origin, body size, schema, ranges, and mathematical constraints.

## Computation and explanation

An allowlisted algorithm computes values. The API returns results, method notes, and numerical limitations. The interface renders tables or plots. JSON export preserves the input and result.

No uploaded program or unrestricted mathematical expression is executed.

# Regression

## The method

Linear, quadratic, and cubic models fit one numeric feature. A fixed-seed shuffle creates the train/test split. Mean and scale come only from training rows. Householder QR solves the least-squares problem.

## The evidence

Held-out MAE, MSE, RMSE, and R-squared describe test errors. Users can inspect predictions, residual plots, and coefficients in standardized coordinates. Comparison keeps the same split across degrees.

Choosing a model repeatedly from test scores reuses test information. Separate validation is still needed.

# Binary classification

## Logistic regression

A sigmoid converts a linear score into a probability. Full-batch gradient descent minimizes penalized cross-entropy. Stratified splitting preserves both binary classes, and scaling uses training data only.

## Evaluation and interpretation

A threshold turns probabilities into labels. The lab reports a confusion matrix, precision, recall, F1, ROC-AUC, Brier score, log-loss, and a majority baseline. A weight changes log-odds in standardized coordinates.

Probabilities are not guaranteed to be calibrated. Associations do not establish causation.

# Clustering and PCA

## K-means

Two-feature observations alternate between nearest-centroid assignment and centroid means. Five deterministic initializations reduce initialization dependence. Inertia and silhouette describe the resulting geometry.

## Principal components

PCA centers data, optionally standardizes features, and diagonalizes a two-by-two sample covariance matrix. Component scores and variance ratios show the projection. Rank-one reconstruction exposes discarded information.

These are descriptive analyses of the supplied data, not held-out classification accuracy.

# The mathematical workbench

## Algebra and calculus

Polynomial simplification, derivatives, and antiderivatives
Real linear and quadratic roots
Bisection with a sign-changing bracket and interval error bound
Definite polynomial integrals

## Linear algebra and statistics

Matrix transpose and multiplication
Determinants, inverses, and AX = B up to 6 by 6
Means, medians, interpolated quartiles, sample variance, and standard deviation
Pearson correlation with constant-column handling

# A worked matrix example

## Problem

A = [[4, 2], [1, 3]]
B = [[8], [7]]
Solve AX = B.
The system is 4x + 2y = 8 and x + 3y = 7.

## Result and check

The solution is x = 1, y = 2.
Substitution gives 4 + 4 = 8 and 1 + 6 = 7.
The algorithm uses partial pivoting and reports a residual.
The determinant is 10, so this fixture is nonsingular.

# Research and scientific integrity

## Where records come from

The curated collection links to arXiv, the International Mathematical Union, Clay Mathematics Institute, and historical references. A bounded arXiv API adapter retrieves recent submissions by category and page.

## How uncertainty appears

Preprints retain a preprint label. Publication and retrieval dates are separate. Timeouts, cache age, and unavailable providers have visible states. A numerical experiment or AI suggestion is never labeled a verified proof.

The live feed is one source. Comprehensive discovery verification and a research knowledge graph remain future work.

# Learning and progress

## Lessons and puzzles

Nine lessons combine an intuitive explanation, formula, worked example, misconception, and checked exercise. Ten puzzle templates cover numbers, algebra, geometry, logic, probability, sequences, calculus, and related topics.

## What the browser remembers

Puzzle progress, streaks, bookmarks, and up to twenty experiment metadata records stay in local storage. Raw uploaded datasets are not automatically persisted. Explicit result downloads may contain the supplied numeric data.

Hints and full solutions support learning. Repeating a completed puzzle does not award duplicate points.

# Authentication and privacy

## Optional Supabase identity

The app includes sign-in, signup, account, recovery, password update, refresh, and sign-out routes. The server verifies user identity with Supabase and stores session tokens in HttpOnly cookies. PKCE binds email confirmation to the requesting browser.

## Owner configuration required

SUPABASE_URL and SUPABASE_PUBLISHABLE_KEY must be configured in Vercel. Supabase also needs the exact callback allowlist and email delivery settings. Live signup and recovery remain unverified until setup is complete.

Account identity does not synchronize local puzzle progress or notebooks. Self-service account deletion is unfinished.

# Security boundaries

## Implemented boundaries

No unrestricted code evaluation
Allowlisted algorithms and bounded input sizes
Same-origin request checks and content-type validation
CSP, output escaping, and per-instance quotas
No password, token, or request-body logging

## Production work still needed

Independent security and accessibility review
Coordinated quotas across serverless instances
Broader concurrency and recovery testing
Owner privacy support and reporting channels
An explicit project license decision

# Development and deployment

## Local development

Node.js 24 to 26
npm start
Open http://127.0.0.1:4173
No production npm dependencies in the web core
Optional provider variables are listed in .env.example

## Vercel deployment

Static browser assets build into dist/public.
/api requests reach a shared Node function.
The live site is mathlens-wine.vercel.app.
Health and numerical endpoints have HTTPS smoke checks.
Native clients use this hosted service.

# Verification

## Verified numerical core

Forty-nine tests passed in the refinement pass. Fixtures check parser rejection, identities, train/test isolation, QR recovery, logistic metrics, clustering, PCA, matrix residuals, roots, statistics, and provider failures.

## Verification scope

Windows and Linux web CI plus a security scan passed for the refinement commit. Production math APIs and actual arXiv retrieval passed. New rendered views still need browser verification. Native checks are recorded separately in their build workflow.

Test results describe executed checks. They do not certify every workflow or every device.

# Applications and releases

## Desktop and Android clients

Windows packages use Electron and an unsigned installer. Linux uses an AppImage. Android uses a WebView APK with debug signing for testing. Each opens the hosted workspace and requires internet.

## Release gates and limits

The native workflow must build real files and pass platform smoke checks before publication. Checksums identify the outputs. Android file exports use the browser version. Email verification should use a regular browser. iOS remains Coming Soon.

The latest GitHub release and its build receipts are the authority for current artifact availability.

# Remaining work

## Mathematics and models

General safe symbolic functions
Decision trees, forests, boosting, and neural networks
Validated attribution and uncertainty methods
Broader datasets and multifeature workflows

## Platform maturity

Account data lifecycle and cloud synchronization
Durable jobs and shared provider cache
Complete localization and strict typing
Rendered accessibility and cross-device testing
Signed production packages and iOS distribution

# Project resources

The running app and the source explain the current implementation

Live application
https://mathlens-wine.vercel.app

Source, build instructions, and release downloads
github.com/manikantbindass/MathInsightAI-Explainable-Math-AI-Model-Analyzer

Core references
README.md / API.md / ARCHITECTURE.md
AUTHENTICATION.md / DISTRIBUTION.md / REFINEMENT_AUDIT.md